

Symbols & Metaprogramming Basics

JavaScript-এর বেশিরভাগ code আমরা normal application logic লেখার জন্য ব্যবহার করি।

কিন্তু JavaScript আমাদের language-এর কিছু **built-in behavior customize, intercept** বা **control** করার সুযোগও দেয়।

এই ধারণাকে বলা হয়:

Metaprogramming

এই section-এ আমরা শিখব:

1. Symbol
2. Symbol.for() এবং Symbol.keyFor()
3. Well-known Symbols
4. Symbol.toPrimitive
5. Symbol.hasInstance
6. Symbol.species
7. Proxy
8. Reflect
9. Proxy traps
10. get, set, apply
11. Tagged template literals

1. Metaprogramming কী?

সহজ ভাষায়:

Program-এর behavior নিয়ে program লেখা = Metaprogramming

Normal programming:

```
const user = {  
  name: "Shariar"  
};  
  
console.log(user.name);
```

এখানে আমরা data নিয়ে কাজ করছি।

Metaprogramming:

```
const user = new Proxy(
  {
    name: "Sharlar"
  },
  {
    get(target, property) {
      console.log(`Reading ${property}`);

      return target[property];
    }
  }
);
```

এখানে আমরা object-এর **property access behavior** intercept করছি।

অর্থাৎ:

```
Normal programming
  ↓
Work with data

Metaprogramming
  ↓
Control/observe language behavior
```

2. Symbol কী?

Symbol হলো JavaScript-এর একটি **primitive data type**।

```
const id = Symbol();
```

এখানে:

```
typeof id
```

হবে:

```
"symbol"
```

3. Symbol-এর মূল বৈশিষ্ট্য

প্রতিটি নতুন Symbol unique।

```
const a = Symbol();  
const b = Symbol();  
  
console.log(a === b);
```

Output:

```
false
```

যদিও:

```
Symbol() === Symbol()
```

লিখলেও result:

```
false
```

কারণ প্রতিটি Symbol আলাদা identity তৈরি করে।

4. Symbol Description

Symbol-এর সাথে একটি description দেওয়া যায়:

```
const id = Symbol("userId");  
  
console.log(id);
```

Conceptually:

```
Symbol(userId)
```

কিন্তু description identity define করে না।

```
const a = Symbol("id");  
const b = Symbol("id");  
  
console.log(a === b);
```

Output:

false

অর্থাৎ:

Same description
≠
Same Symbol

5. Symbol-এর Type

Symbol primitive:

```
const id = Symbol("id");  
  
console.log(typeof id);
```

Output:

symbol

Primitive types-এর মধ্যে:

```
string  
number  
bigint  
boolean  
undefined  
symbol  
null
```

6. Symbol দিয়ে Unique Object Keys

Symbol-এর সবচেয়ে common use হলো object-এর unique property key তৈরি করা।

```
const id = Symbol("id");

const user = {
  name: "Shariar",
  [id]: 101
};
```

এখানে:

```
user[id]
```

হবে:

```
101
```

7. কেন Symbol Key দরকার?

ধরি একটি object-এর property:

```
const user = {
  id: 101
};
```

অন্য কোনো library একই object-এ:

```
user.id = 999;
```

করলে collision হতে পারে।

Symbol ব্যবহার করলে:

```
const libraryId = Symbol("id");  
  
user[libraryId] = 999;
```

এটি আলাদা property।

```
"id"
```

এবং:

```
Symbol("id")
```

দুটি আলাদা key।

8. Symbol Keys-এর গুরুত্বপূর্ণ বৈশিষ্ট্য

Symbol-keyed properties সাধারণত:

```
Object.keys()
```

দিয়ে পাওয়া যায় না।

Example:

```
const id = Symbol("id");  
  
const user = {  
  name: "Sharir",  
  [id]: 101  
};  
  
console.log(Object.keys(user));
```

Output:

```
["name"]
```

Symbol key এখানে নেই।

9. Symbol Keys কীভাবে পাওয়া যায়?

ব্যবহার করতে পারো:

```
Object.getOwnPropertySymbols()
```

Example:

```
const id = Symbol("id");

const user = {
  name: "Sharlar",
  [id]: 101
};

console.log(Object.getOwnPropertySymbols(user));
```

এটি Symbol keys-এর array return করবে।

10. Reflect.ownKeys()

Object-এর string এবং Symbol—দুই ধরনের own keys একসাথে পেতে:

```
const id = Symbol("id");

const user = {
  name: "Sharlar",
  [id]: 101
};

console.log(Reflect.ownKeys(user));
```

এখানে:

```
"name"
Symbol("id")
```

দুটিই পাওয়া যাবে।

11. `for...in` এবং **Symbol**

`for...in` Symbol-keyed properties iterate করে না।

```
const id = Symbol("id");

const user = {
  name: "Shariar",
  [id]: 101
};

for (const key in user) {
  console.log(key);
}
```

Output:

```
name
```

Symbol key এখানে আসবে না।

12. Symbol Property Private কি?

না।

এটা খুব important।

Symbol key:

```
const secret = Symbol("secret");

const user = {
  [secret]: "hidden"
};
```

সরাসরি জানা না গেলেও property পুরোপুরি private নয়।

কারণ:

```
Object.getOwnPropertySymbols(user)
```

দিয়ে Symbol key discover করা যায়।

তাই:

Symbol = unique/less collision-prone key

Symbol $\neq$ true private property

True private field-এর জন্য:

```
class User {  
  #password;  
}
```

ব্যবহার করা হয়।

13. Symbol as Computed Property

Object literal-এ Symbol key দিতে bracket notation দরকার:

```
const id = Symbol("id");  
  
const user = {  
  [id]: 123  
};
```

ভুল:

```
const user = {  
  id: 123  
};
```

এটি Symbol key নয়।

14. Symbol as Object Key

```
const id = Symbol("id");

const user = {};

user[id] = 101;

console.log(user[id]);
```

Output:

```
101
```

15. Symbol as Map Key

Symbol শুধু Object key হিসেবেই নয়, Map key হিসেবেও ব্যবহার করা যায়।

```
const key = Symbol("key");

const map = new Map();

map.set(key, "value");

console.log(map.get(key));
```

16. Symbol.for()

`Symbol()` প্রতিবার নতুন Symbol তৈরি করে।

```
const a = Symbol("id");
const b = Symbol("id");

console.log(a === b);
```

Output:

```
false
```

কিন্তু `Symbol.for()` একটি **global symbol registry** ব্যবহার করে।

```
const a = Symbol.for("id");  
const b = Symbol.for("id");  
  
console.log(a === b);
```

Output:

```
true
```

17. Symbol Registry

Conceptually:

Global Symbol Registry

```
"id" → Symbol("id")  
"user" → Symbol("user")  
"app" → Symbol("app")
```

যখন:

```
Symbol.for("id")
```

call করা হয়, registry-তে `"id"` থাকলে একই Symbol পাওয়া যায়।

না থাকলে নতুন Symbol তৈরি করে registry-তে রাখে।

18. `Symbol()` vs `Symbol.for()`

<code>Symbol()</code>	<code>Symbol.for()</code>
প্রতিবার নতুন Symbol	Registry থেকে shared Symbol

Symbol()	Symbol.for()
Global registry ব্যবহার করে না	Global Symbol Registry ব্যবহার করে
Same description হলেও আলাদা	Same key হলে same Symbol
Local unique identity	Shared identity

Example:

```
Symbol("id") === Symbol("id")
```

```
false
```

কিন্তু:

```
Symbol.for("id") === Symbol.for("id")
```

```
true
```

19. Symbol.keyFor()

Symbol.for() দিয়ে তৈরি global Symbol-এর registry key বের করতে:

```
const id = Symbol.for("userId");
console.log(Symbol.keyFor(id));
```

Output:

```
"userId"
```

20. `Symbol.keyFor()` এবং **Normal Symbol**

```
const id = Symbol("userId");  
  
console.log(Symbol.keyFor(id));
```

Output:

```
undefined
```

কারণ:

```
Symbol("userId")
```

global registry-তে registered নয়।

21. Important Comparison

```
const local = Symbol("id");  
const global = Symbol.for("id");  
  
console.log(Symbol.keyFor(local));  
console.log(Symbol.keyFor(global));
```

Output:

```
undefined  
id
```

22. Well-known Symbols

JavaScript কিছু predefined Symbol দিয়েছে যেগুলো language behavior customize করতে ব্যবহৃত হয়।

এগুলোকে বলা হয়:

Well-known Symbols

Examples:

```
Symbol.iterator  
Symbol.toPrimitive  
Symbol.hasInstance  
Symbol.species  
Symbol.toStringTag  
Symbol.asyncIterator  
Symbol.match  
Symbol.replace  
Symbol.search  
Symbol.split  
Symbol.isConcatSpreadable
```

এই section-এ আমরা বিশেষভাবে দেখব:

- `Symbol.toPrimitive`
- `Symbol.hasInstance`
- `Symbol.species`

23. `Symbol.iterator`

`Symbol.iterator` iterable protocol define করে।

Example:

```
const numbers = {  
  *[Symbol.iterator]() {  
    yield 1;  
    yield 2;  
    yield 3;  
  }  
};
```

এখন:

```
for (const number of numbers) {  
  console.log(number);  
}
```

এখানে `Symbol.iterator` object-এর iteration behavior define করছে।

24. `Symbol.toPrimitive`

`Symbol.toPrimitive` object কীভাবে primitive value-তে convert হবে সেটা customize করতে দেয়।

Example:

```
const user = {
  value: 100,

  [Symbol.toPrimitive](hint) {
    if (hint === "number") {
      return this.value;
    }

    return "User";
  }
};
```

এখন:

```
console.log(Number(user));
```

Output:

```
100
```

আর:

```
console.log(String(user));
```

Output:

```
User
```

25. Symbol.toPrimitive Hint

JavaScript conversion-এর সময় hint দিতে পারে:

```
"number"  
"string"  
"default"
```

Example:

```
const obj = {  
  [Symbol.toPrimitive](hint) {  
    console.log(hint);  
  
    return 10;  
  }  
};
```

26. Number Conversion

```
const obj = {  
  [Symbol.toPrimitive](hint) {  
    console.log(hint);  
  
    return 10;  
  }  
};  
  
console.log(Number(obj));
```

এখানে hint হবে:

```
number
```

27. String Conversion

```
const obj = {
  [Symbol.toPrimitive](hint) {
    console.log(hint);

    return "hello";
  }
};

console.log(String(obj));
```

এখানে hint:

```
string
```

28. Default Hint

Operator behavior-এর ক্ষেত্রে "default" hint আসতে পারে।

Example:

```
const obj = {
  [Symbol.toPrimitive](hint) {
    console.log(hint);

    return 10;
  }
};

console.log(obj + 5);
```

এখানে primitive conversion-এর সময় default hint ব্যবহার হতে পারে।

29. `Symbol.toPrimitive` Return অবশ্যই **Primitive**

হতে হবে

ভুল:

```
const obj = {  
  [Symbol.toPrimitive]() {  
    return {};  
  }  
};
```

কারণ `Symbol.toPrimitive` method-কে primitive value return করতে হয়।

Valid:

```
return 10;
```

অথবা:

```
return "hello";
```

অথবা:

```
return true;
```

30. `Symbol.toPrimitive` Conversion Priority

Object → primitive conversion-এ `Symbol.toPrimitive` থাকলে এটি প্রথম সুযোগ পায়।

Conceptually:

```
Object conversion  
  ↓  
Symbol.toPrimitive?  
  |  
Yes  
  ↓
```

```
Use it
  |
  No
  ↓
valueOf()/toString()
```

31. Symbol.hasInstance

Symbol.hasInstance customize করে:

```
instanceof
```

operator-এর behavior ।

Normal:

```
value instanceof Constructor
```

Conceptually constructor-এর:

```
Constructor[Symbol.hasInstance](value)
```

behavior-এর সাথে সম্পর্কিত ।

32. Normal instanceof

```
class User {}

const user = new User();

console.log(user instanceof User);
```

Output:

```
true
```

33. Custom Symbol.hasInstance

আমরা নিজের class-এর `instanceof` behavior customize করতে পারি।

```
class User {  
  static [Symbol.hasInstance](value) {  
    return value && value.role === "user";  
  }  
}
```

এখন:

```
const object = {  
  role: "user"  
};  
  
console.log(object instanceof User);
```

Output:

```
true
```

যদিও:

```
object
```

আসলে `User` class দিয়ে তৈরি হয়নি।

34. Symbol.hasInstance Mental Model

Normal:

```
object instanceof User  
  ↓  
User[Symbol.hasInstance](object)
```

Custom implementation:

```
class User {  
  static [Symbol.hasInstance](value) {  
    return value?.role === "user";  
  }  
}
```

এখন `instanceof` custom rule follow করছে।

35. `Symbol.species`

`Symbol.species` একটু advanced।

এটি বিশেষভাবে built-in class-এর methods যখন নতুন object তৈরি করে, তখন কোন **constructor** ব্যবহার হবে সেটা influence করতে পারে।

সবচেয়ে common example:

Array

এবং subclassing।

36. Array Subclass

```
class MyArray extends Array {}
```

এখন:

```
const numbers = new MyArray(1, 2, 3);  
  
const result = numbers.map(x => x * 2);  
  
console.log(result instanceof MyArray);
```

Subclass behavior-এর কারণে derived array methods result-এর constructor selection-এর সাথে `Symbol.species` সম্পর্কিত।

37. Customizing Array Species

```
class MyArray extends Array {  
  static get [Symbol.species]() {  
    return Array;  
  }  
}
```

এখন array methods নতুন result তৈরি করার সময় `Array` constructor ব্যবহার করতে পারে।

```
const numbers = new MyArray(1, 2, 3);  
  
const result = numbers.map(x => x * 2);  
  
console.log(result instanceof Array);  
console.log(result instanceof MyArray);
```

Expected:

```
true  
false
```

38. `Symbol.species` কেন দরকার?

ধরি:

```
class MyArray extends Array {}
```

আমরা চাই:

```
map()  
filter()  
slice()
```

এর মতো methods নতুন result তৈরি করলেও সেটা subclass হোক বা plain `Array` হোক—এই behavior control করতে `Symbol.species` ব্যবহার করা যায়।

39. `Symbol.species` Conceptual Flow

```
MyArray instance
  ↓
map()
  ↓
Create new array-like result
  ↓
Which constructor?
  ↓
Symbol.species
  ↓
Array / MyArray / custom constructor
```

40. Proxy কী?

`Proxy` হলো এমন একটি object wrapper যা অন্য object-এর operations **intercept** করতে পারে।

Basic:

```
const target = {
  name: "Sharlar"
};

const proxy = new Proxy(target, {});
```

এখন:

```
console.log(proxy.name);
```

Proxy target-এর behavior-এর সামনে একটি interception layer তৈরি করে।

41. Proxy Structure

```
const proxy = new Proxy(target, handler);
```

দুটি অংশ:

Target

যে object-কে wrap করা হচ্ছে।

target

Handler

কোন operation intercept হবে তার rules:

handler

Mental model:

```
Code
↓
Proxy
↓
Handler / Trap
↓
Target
```

42. Trap কী?

Proxy-এর handler-এর special methods-কে বলা হয়:

Traps

Examples:

```
get
set
has
deleteProperty
apply
construct
ownKeys
getOwnPropertyDescriptor
defineProperty
getPrototypeOf
```

```
setPrototypeOf  
isExtensible  
preventExtensions
```

এই section-এ focus:

- get
- set
- apply

43. Proxy `get` Trap

`get` property access intercept করে।

```
const user = {  
  name: "Shariar"  
};  
  
const proxy = new Proxy(user, {  
  get(target, property) {  
    console.log(`Reading: ${property}`);  
  
    return target[property];  
  }  
});
```

এখন:

```
console.log(proxy.name);
```

Output conceptually:

```
Reading: name  
Shariar
```

44. `get` Trap Parameters

```
get(target, property, receiver)
```

`target`

Original object।

`property`

যে property access হচ্ছে।

`receiver`

Proxy বা actual receiver object।

Example:

```
get(target, property, receiver) {  
  console.log(target);  
  console.log(property);  
  console.log(receiver);  
  
  return Reflect.get(target, property, receiver);  
}
```

45. কেন `Reflect.get()` ব্যবহার করা ভালো?

এভাবে করা যায়:

```
return target[property];
```

কিন্তু Proxy-এর ভিতরে সাধারণ default behavior preserve করতে:

```
return Reflect.get(target, property, receiver);
```

অনেক ক্ষেত্রে cleaner।

46. Proxy `set` Trap

`set` property assignment intercept করে।

```
const user = {
  name: "Shariar"
};

const proxy = new Proxy(user, {
  set(target, property, value) {
    console.log(
      `Setting ${property} = ${value}`
    );

    target[property] = value;

    return true;
  }
});
```

এখন:

```
proxy.name = "Duke";
```

Output:

```
Setting name = Duke
```

47. `set` Trap-এর Return Value

`set` trap সাধারণত boolean-like success signal return করবে।

```
return true;
```

মানে assignment সফল।

ভুলভাবে:

```
return false;
```

করলে strict mode-এ assignment-related error হতে পারে।

48. Validation with `set`

Proxy-এর practical use case:

```
const user = {
  age: 30
};

const proxy = new Proxy(user, {
  set(target, property, value) {
    if (property === "age") {
      if (typeof value !== "number") {
        throw new TypeError("Age must be a number");
      }

      if (value < 0) {
        throw new RangeError("Age cannot be negative");
      }
    }

    return Reflect.set(
      target,
      property,
      value
    );
  }
});
```

এখন:

```
proxy.age = 31;
```

valid।

কিন্তু:

```
proxy.age = "31";
```

error।

49. Proxy `get` দিয়ে Default Value

```
const user = {  
  name: "Sharir"  
};  
  
const proxy = new Proxy(user, {  
  get(target, property) {  
    return property in target  
      ? target[property]  
      : "Not Found";  
  }  
});
```

এখন:

```
console.log(proxy.name);  
console.log(proxy.age);
```

Output:

```
Sharir  
Not Found
```

50. Proxy `apply` Trap

`apply` trap function call intercept করে।

Proxy শুধু objects-এর জন্য নয়—**functions**-ও **target** হতে পারে।

```
function greet(name) {  
  return `Hello ${name}`;  
}
```

```
const proxy = new Proxy(greet, {
  apply(target, thisArg, args) {
    console.log("Function called");

    return target.apply(thisArg, args);
  }
});
```

এখন:

```
proxy("Shariar");
```

Output conceptually:

```
Function called
Hello Shariar
```

51. `apply` Trap Parameters

```
apply(target, thisArg, argumentsList)
```

target

Original function ।

thisArg

Function call-এর `this` value ।

argumentsList

Arguments-এর array ।

Example:

```
apply(target, thisArg, args) {
  console.log(args);

  return Reflect.apply(
```

```
    target,  
    thisArg,  
    args  
  );  
}
```

52. Reflect.apply()

Function call-এর default behavior preserve করতে:

```
Reflect.apply(  
  target,  
  thisArg,  
  args  
);
```

ব্যবহার করা যায়।

এটি conceptually:

```
target.apply(thisArg, args)
```

এর সাথে related।

53. Proxy + Logging

```
const user = {  
  name: "Sharlar",  
  age: 31  
};  
  
const proxy = new Proxy(user, {  
  get(target, property, receiver) {  
    console.log(`GET: ${String(property)}`);  
  
    return Reflect.get(  
      target,  
      property,  
      receiver  
    );  
  }  
});
```

```

    );
  },

  set(target, property, value, receiver) {
    console.log(
      `SET: ${String(property)} = ${value}`
    );

    return Reflect.set(
      target,
      property,
      value,
      receiver
    );
  }
});

```

এখন:

```

proxy.name;
proxy.age = 32;

```

দুটো operation observe করা যাবে।

54. Reflect কী?

Reflect হলো JavaScript-এর built-in object যেটি অনেক fundamental object operations-এর **method-based API** দেয়।

Examples:

```

Reflect.get()
Reflect.set()
Reflect.has()
Reflect.deleteProperty()
Reflect.ownKeys()
Reflect.apply()
Reflect.construct()
Reflect.defineProperty()

```

Proxy-এর সাথে **Reflect** খুব common combination।

55. Proxy + Reflect Mental Model

```
Operation
  ↓
Proxy Trap
  ↓
Custom Logic
  ↓
Reflect operation
  ↓
Target
```

Example:

```
get(target, property, receiver) {
  console.log("GET");

  return Reflect.get(
    target,
    property,
    receiver
  );
}
```

56. Why Reflect Instead of Direct Target Access?

ধরি:

```
return target[property];
```

এতে basic access কাজ করবে।

কিন্তু:

```
Reflect.get(
  target,
  property,
```

```
    receiver
  );
```

ব্যবহার করলে receiver-aware semantics preserve করা সহজ হয়।

বিশেষ করে:

- getters
- prototype inheritance
- Proxy layering

এর মতো advanced cases-এ `Reflect` useful।

57. `Reflect.set()`

```
Reflect.set(
  target,
  property,
  value,
  receiver
);
```

Proxy `set` trap-এ:

```
set(target, property, value, receiver) {
  return Reflect.set(
    target,
    property,
    value,
    receiver
  );
}
```

এটি default assignment behavior delegate করার clean pattern।

58. `Reflect.get()`

```
const user = {
  name: "Shariar"
};
```

```
console.log(  
  Reflect.get(user, "name")  
);
```

Output:

```
Shariar
```

59. Reflect.has()

Equivalent-style operation:

```
Reflect.has(object, property)
```

Example:

```
const user = {  
  name: "Shariar"  
};  
  
console.log(  
  Reflect.has(user, "name")  
);
```

Output:

```
true
```

60. Reflect.deleteProperty()

```
const user = {  
  name: "Shariar"  
};  
  
Reflect.deleteProperty(user, "name");
```

```
console.log(user.name);
```

Output:

```
undefined
```

61. Reflect.ownKeys()

String + Symbol own keys:

```
const id = Symbol("id");

const user = {
  name: "Sharir",
  [id]: 101
};

console.log(
  Reflect.ownKeys(user)
);
```

দুই ধরনের key পাওয়া যাবে।

62. Proxy vs Reflect

Proxy	Reflect
Operations intercept/customize করে	Operations perform করার API
Handler + traps	Static methods
Behavior modify করতে পারে	Default semantics delegate করতে useful
Wrapper	Operation utility
<code>get</code> , <code>set</code> , <code>apply</code> traps	<code>get</code> , <code>set</code> , <code>apply</code> methods

একসাথে সবচেয়ে powerful:

```
Proxy = intercept  
Reflect = perform/delegate
```

63. Proxy দিয়ে Read-only Object

```
const user = {  
  name: "Sharlar",  
  age: 31  
};  
  
const readOnlyUser = new Proxy(user, {  
  set() {  
    throw new Error(  
      "Object is read-only"  
    );  
  }  
});
```

এখন:

```
readOnlyUser.age = 32;
```

error হবে।

64. Proxy দিয়ে Access Control

```
const user = {  
  name: "Sharlar",  
  password: "secret"  
};  
  
const safeUser = new Proxy(user, {  
  get(target, property) {  
    if (property === "password") {  
      return undefined;  
    }  
  
    return Reflect.get(target, property);  
  }  
});
```

```
}  
});
```

এখন:

```
console.log(safeUser.name);  
console.log(safeUser.password);
```

Output conceptually:

```
Shariar  
undefined
```

65. Proxy দিয়ে Negative Array Index

JavaScript Array-এ:

```
array[-1]
```

সাধারণভাবে last element দেয় না।

Proxy দিয়ে custom behavior বানানো যায়:

```
const numbers = [10, 20, 30];  
  
const proxy = new Proxy(numbers, {  
  get(target, property, receiver) {  
    if (  
      typeof property === "string" &&  
      /^-\d+$/.test(property)  
    ) {  
      const index = Number(property);  
  
      return target[  
        target.length + index  
      ];  
    }  
  
    return Reflect.get(  
      target,
```

```
        property,  
        receiver  
    );  
}  
});
```

এখন:

```
console.log(proxy[-1]);
```

Output:

```
30
```

এটি metaprogramming-এর practical example ।

66. Tagged Template Literals

Template literals আমরা আগে দেখেছি:

```
const name = "Sharir";  
const message = `Hello ${name}`;
```

Tagged template literal হলো এমন template literal যেখানে একটি function template-এর আগে বসে:

```
tag`Hello ${name}`;
```

এখানে:

```
tag
```

হলো tag function ।

67. Basic Tagged Template

```
function tag(strings, ...values) {  
  console.log(strings);  
  console.log(values);  
}  
  
const name = "Shariar";  
  
tag`Hello ${name}!`;
```

Tag function template literal-এর raw structure এবং interpolated values পায়।

68. Tagged Template-এর Arguments

এই template:

```
tag`Hello ${name}, you are ${age}`;
```

Tag function roughly পাবে:

```
strings  
values
```

`strings`:

```
[  
  "Hello ",  
  ", you are ",  
  ""  
]
```

`values`:

```
[  
  name,  
  age  
]
```

69. Simple Tag Function

```
function tag(strings, ...values) {  
  return "Custom output";  
}  
  
const result = tag`Hello ${"Shariar"}`;  
  
console.log(result);
```

Output:

```
Custom output
```

Tag function পুরো template processing-এর control নেয়।

70. Reconstructing Template

```
function tag(strings, ...values) {  
  let result = strings[0];  
  
  for (let i = 0; i < values.length; i++) {  
    result += values[i];  
    result += strings[i + 1];  
  }  
  
  return result;  
}
```

ব্যবহার:

```
const name = "Shariar";  
const age = 31;  
  
console.log(  
  tag`Name: ${name}, Age: ${age}`  
);
```

Output:

Name: Shariar, Age: 31

71. Tagged Template কেন Useful?

Tagged templates useful হতে পারে:

- custom formatting
- escaping
- localization
- SQL-like DSLs
- CSS-in-JS
- HTML templating
- sanitization
- logging
- query builders

72. Tagged Template দিয়ে Formatting

```
function upper(strings, ...values) {  
  let result = strings[0];  
  
  for (let i = 0; i < values.length; i++) {  
    result += String(values[i]).toUpperCase();  
    result += strings[i + 1];  
  }  
  
  return result.toUpperCase();  
}
```

ব্যবহার:

```
const name = "Shariar";  
  
console.log(  
  upper`Hello ${name}`  
);
```

73. Tagged Template দিয়ে HTML Escaping

একটি simplified escaping example:

```
function escapeHTML(value) {
  return String(value)
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("'", "&quot;")
    .replaceAll('"', "&#39;");
}

function html(strings, ...values) {
  let result = strings[0];

  for (let i = 0; i < values.length; i++) {
    result += escapeHTML(values[i]);
    result += strings[i + 1];
  }

  return result;
}
```

ব্যবহার:

```
const username = "<script>alert(1)</script>";

const output = html`
  <h1>Hello ${username}</h1>
`;
```

এখানে interpolated value escape করা হচ্ছে।

Production security-এর জন্য framework-এর built-in escaping/sanitization ব্যবহার করাই safer। Tagged templates দিয়ে security system নিজে বানালে অনেক edge case miss হতে পারে।

74. String.raw

Tagged templates-এর সাথে related একটি built-in tag:

```
String.raw
```

Example:

```
const path = String.raw`C:\Users\Shariar`;
```

এটি backslash interpretation-এর ক্ষেত্রে useful।

75. Tagged Template vs Normal Template Literal

Normal

```
const message = `Hello ${name}`;
```

JavaScript নিজেই interpolation process করে।

Tagged

```
tag`Hello ${name}`;
```

এখানে:

```
tag function
```

template processing control করে।

76. Tagged Template-এর Raw Strings

Tag function-এর `strings` object-এর `.raw` property থাকে।

Example:

```
function tag(strings) {  
  console.log(strings.raw);  
}
```

```
tag`Hello\nWorld`;
```

`raw` cooked escape processing-এর আগের template chunks access করার জন্য useful ।

77. Symbol + Proxy + Reflect — How They Relate

এই তিনটা metaprogramming-এর different tools:

```
Symbol
  ↓
Special keys / language hooks

Proxy
  ↓
Intercept operations

Reflect
  ↓
Perform/delegate operations
```

Well-known Symbols Proxy-এর মতো mechanisms-এর সাথে language behavior customize করার broader ecosystem-এর অংশ ।

78. Proxy Trap এবং Reflect Mapping

কিছু common pair:

Proxy Trap	Reflect Method
<code>get</code>	<code>Reflect.get()</code>
<code>set</code>	<code>Reflect.set()</code>
<code>has</code>	<code>Reflect.has()</code>
<code>deleteProperty</code>	<code>Reflect.deleteProperty()</code>
<code>ownKeys</code>	<code>Reflect.ownKeys()</code>
<code>apply</code>	<code>Reflect.apply()</code>
<code>construct</code>	<code>Reflect.construct()</code>

Proxy Trap

Reflect Method

defineProperty

Reflect.defineProperty()

Pattern:

```
const proxy = new Proxy(target, {
  get(target, property, receiver) {
    // custom behavior

    return Reflect.get(
      target,
      property,
      receiver
    );
  }
});
```

79. Proxy-এর Important Rule

Proxy magic হলোও সব JavaScript rule bypass করা যায় না।

Proxy traps-কে JavaScript-এর **invariants** respect করতে হয়।

বিশেষ করে:

- non-configurable properties
- non-writable properties
- extensibility constraints

এর মতো ক্ষেত্রে trap arbitrary result দিতে পারে না।

80. Proxy Performance Consideration

Proxy powerful, কিন্তু normal direct property access-এর তুলনায় additional abstraction layer তৈরি করে।

তাই:

Proxy

কে শুধু "সব object-এ use করব" এমন pattern বানানো উচিত নয়।

Use when:

- interception দরকার
 - validation দরকার
 - reactive behavior দরকার
 - debugging/logging দরকার
 - dynamic API দরকার
-

81. Proxy-এর Common Use Cases

Real-world libraries/frameworks-এ Proxy দিয়ে করা যেতে পারে:

- validation
 - access control
 - logging
 - reactive state
 - lazy property access
 - API wrappers
 - mocking
 - observable objects
 - virtual properties
 - read-only views
-

82. Proxy বনাম Getter/Setter

Getter/setter:

```
const user = {  
  get name() {  
    return "Shariar";  
  }  
};
```

এখানে specific property behavior customize হচ্ছে।

Proxy:

```
const proxy = new Proxy(user, {  
  get(target, property) {
```

```
    // any property
  }
});
```

এখানে dynamically অনেক property access intercept করা যায়।

Simple rule

```
Specific property behavior
  ↓
Getter / Setter
  ↓
General object operation interception
  ↓
Proxy
```

83. Symbol বনাম String Property

String key:

```
const user = {
  id: 1
};
```

Potential collision:

```
user.id
```

Symbol key:

```
const id = Symbol("id");

user[id] = 1;
```

Unique identity:

```
String key
  ↓
"name"
```

```
Symbol key
  ↓
Symbol("name")
```

একই description হলেও Symbol identity আলাদা হতে পারে।

84. `Symbol.for()` কখন ব্যবহার করব?

`Symbol.for()` useful যখন application-এর বিভিন্ন জায়গায় একই Symbol identity share করতে হবে।

Example:

```
const EVENT = Symbol.for("app.event");
```

অন্য module-এ:

```
const EVENT = Symbol.for("app.event");
```

দুটো same registry Symbol পাবে।

তবে সাধারণ unique internal key-এর জন্য:

```
Symbol()
```

অনেক সময় যথেষ্ট।

85. Symbol Naming Convention

Library/internal symbols-এর description meaningful রাখা ভালো:

```
Symbol("cacheKey")
```

বা registry key:

```
Symbol.for("myApp.cacheKey")
```

এতে debugging সহজ হয়।

86. Well-known Symbols — Broader List

এই section-এর বাইরে আরও useful well-known Symbols আছে:

Iteration

```
Symbol.iterator  
Symbol.asyncIterator
```

Primitive conversion

```
Symbol.toPrimitive
```

```
instanceof
```

```
Symbol.hasInstance
```

Species

```
Symbol.species
```

String matching

```
Symbol.match  
Symbol.matchAll  
Symbol.replace  
Symbol.search  
Symbol.split
```

Object behavior

```
Symbol.toStringTag  
Symbol.isConcatSpreadable  
Symbol.unscopables
```

সবগুলো মুখস্থ করার চেয়ে কোন **problem solve** করে সেটা বোঝা বেশি important।

87. Symbol.toStringTag

এটি object-এর `Object.prototype.toString()` representation customize করতে পারে।

```
const user = {  
  [Symbol.toStringTag]: "User"  
};  
  
console.log(  
  Object.prototype.toString.call(user)  
);
```

Conceptually output:

```
[object User]
```

88. Symbol.isConcatSpreadable

Array concat-এর behavior influence করতে পারে।

```
const values = [1, 2];  
  
values[Symbol.isConcatSpreadable] = false;  
  
console.log([0].concat(values));
```

এখানে array-টাকে spread না করে single element হিসেবে treat করার behavior পাওয়া যেতে পারে।

এটি advanced well-known Symbol-এর example।

89. Symbol.match

Custom object-কে string matching behavior-এর সাথে integrate করা যায়।

Conceptually:

```
const matcher = {
  [Symbol.match](text) {
    return text.includes("JS")
      ? ["JS"]
      : null;
  }
};
```

এ ধরনের Symbol built-in operations customize করার power দেখায়।

90. Metaprogramming-এর Big Picture

JavaScript-এর built-in protocols অনেক জায়গায় hook দেয়:

```
Object behavior
├── property access
│   └── ↓
│       Proxy
├── iteration
│   └── ↓
│       Symbol.iterator
├── primitive conversion
│   └── ↓
│       Symbol.toPrimitive
├── instanceof
│   └── ↓
│       Symbol.hasInstance
├── derived collection type
│   └── ↓
│       Symbol.species
└── template processing
    └── ↓
        Tagged Templates
```

এটাই metaprogramming mindset।

91. Common Mistakes

Mistake 1: Symbol description unique identity তৈরি করে ভাবা

```
Symbol("id") === Symbol("id")
```

```
false
```

।

Description শুধু descriptive metadata ।

Mistake 2: Symbol property private মনে করা

```
const secret = Symbol("secret");
```

এটা true private নয়।

```
Object.getOwnPropertySymbols()
```

 দিয়ে পাওয়া যায়।

Mistake 3: Symbol() এবং Symbol.for() একই ভাবা

```
Symbol("id") !== Symbol("id")
```

কিন্তু:

```
Symbol.for("id") === Symbol.for("id")
```

Mistake 4: Proxy target modify করলেই Proxy behavior সবসময় আলাদা থাকবে ভাবা

Proxy এবং target একই underlying object share করতে পারে।

```
proxy.name = "Duke";
```

handler target-এ write করলে:

```
target.name
```

ও update হবে।

Mistake 5: `set` trap-এ `boolean return` ভুলে যাওয়া

```
set(target, property, value) {  
  target[property] = value;  
}
```

এর বদলে সাধারণ pattern:

```
return Reflect.set(  
  target,  
  property,  
  value  
);
```

Mistake 6: `get` trap-এ `recursive access`

এই ধরনের ভুল:

```
get(target, property) {  
  return proxy[property];  
}
```

এতে আবার Proxy-এর `get` trap trigger হতে পারে।

সাধারণত:

```
return Reflect.get(  
  target,  
  property,  
  receiver  
);
```

ব্যবহার করো।

Mistake 7: Proxy দিয়ে সবকিছু solve করার চেষ্টা

Proxy powerful কিন্তু unnecessary complexity এবং performance overhead আনতে পারে।

Mistake 8: FinalizationRegistry -এর মতো GC-based behavior-এর ওপর correctness depend করা

GC deterministic নয়।

92. Output Prediction

Question 1

```
const a = Symbol("id");
const b = Symbol("id");

console.log(a === b);
```

Question 2

```
const a = Symbol.for("id");
const b = Symbol.for("id");

console.log(a === b);
console.log(Symbol.keyFor(a));
```

Question 3

```
const id = Symbol("id");

const user = {
  name: "Sharif",
  [id]: 100
};
```

```
console.log(Object.keys(user));  
console.log(Object.getOwnPropertySymbols(user));
```

Question 4

```
const obj = {  
  [Symbol.toPrimitive](hint) {  
    console.log(hint);  
  
    return 10;  
  }  
};  
  
console.log(Number(obj));
```

Hint কী হবে?

Question 5

```
const target = {  
  name: "Shariar"  
};  
  
const proxy = new Proxy(target, {  
  get(target, property) {  
    console.log("GET:", property);  
  
    return target[property];  
  }  
});  
  
console.log(proxy.name);
```

Output order কী হবে?

Question 6

```
function add(a, b) {  
  return a + b;  
}
```

```
const proxy = new Proxy(add, {
  apply(target, thisArg, args) {
    console.log(args);

    return Reflect.apply(
      target,
      thisArg,
      args
    );
  }
});

console.log(proxy(10, 20));
```

93. Practice Tasks

Level 1 — Symbol Key

একটি object বানাও যেখানে:

```
name
age
Symbol-based internal ID
```

থাকবে।

তারপর:

- `Object.keys()`
- `Object.getOwnPropertySymbols()`
- `Reflect.ownKeys()`

তিনটির output compare করো।

Level 2 — Symbol Registry

দুটি জায়গায়:

```
Symbol.for("app.id")
```

ব্যবহার করে **prove** করো যে একই **Symbol** পাওয়া যাচ্ছে।

তারপর:

```
Symbol.keyFor()
```

দিয়ে registry key বের করো।

Level 3 — Symbol.toPrimitive

একটি **Money** object বানাও:

```
const money = new Money(500);
```

যাতে:

```
Number(money)
```

→ 500

এবং:

```
String(money)
```

→ "500 BDT"

return করে।

Level 4 — Custom instanceof

এমন class বানাও:

```
class Admin {}
```

যেখানে:

```
{ role: "admin" } instanceof Admin
```

→ true

কিন্তু:

```
{ role: "user" } instanceof Admin
```

→ false

Level 5 — Proxy Validation

একটি user object Proxy দিয়ে wrap করো।

Rules:

```
name → string  
age → number  
age >= 0
```

Invalid assignment হলে error throw করবে।

Level 6 — Proxy Logger

এমন Proxy বানাও যেটা:

```
GET property  
SET property
```

প্রতিবার console-এ log করবে।

Default behavior-এর জন্য `Reflect.get()` এবং `Reflect.set()` ব্যবহার করো।

Level 7 — Function Proxy

একটি function Proxy দিয়ে wrap করো যেটা:

```
Function called with:  
[arguments]
```

এভাবে log করবে এবং তারপর original function execute করবে।

Level 8 — Tagged Template

এমন tag function বানাও:

```
upper`hello ${name}`
```

যা পুরো output uppercase করে return করবে।

Level 9 — Mini Metaprogramming Challenge

একটি object তৈরি করো:

```
const user = {  
  name: "Sharlar",  
  age: 31  
};
```

তারপর Proxy ব্যবহার করে:

- read log
- write log
- age validation
- unknown property-এর জন্য "Not Found"

implement করো।

94. Interview Questions

1. Symbol কী?

JavaScript-এর একটি primitive type যা unique property keys এবং language-level protocols-এর জন্য ব্যবহৃত হয়।

2. দুইটি `Symbol("id")` কি **equal**?

না।

```
Symbol("id") === Symbol("id")
```

→ `false`

3. `Symbol.for()` কী?

Global Symbol Registry ব্যবহার করে shared Symbol তৈরি/retrieve করে।

4. `Symbol.keyFor()` কী?

`Symbol.for()` দিয়ে registry-তে registered Symbol-এর key return করে।

5. Symbol property কি **private**?

না। Symbol property unique হলেও discoverable।

6. `Symbol.toPrimitive` কী?

Object primitive conversion customize করতে ব্যবহৃত well-known Symbol।

7. `Symbol.hasInstance` কী?

`instanceof` behavior customize করতে ব্যবহৃত হয়।

8. `Symbol.species` কী?

Derived built-in objects-এর methods নতুন result তৈরি করার সময় কোন constructor ব্যবহার করবে সেটা influence করতে পারে।

9. Proxy কী?

একটি object/function-এর operations intercept এবং customize করার wrapper mechanism।

10. Trap কী?

Proxy handler-এর special interception method, যেমন:

```
get  
set  
apply
```

11. Reflect কী?

Fundamental object operations-এর method-based API।

12. Proxy-এর সাথে Reflect কেন ব্যবহার করা হয়?

Custom logic-এর পর default behavior cleanly delegate/preserve করার জন্য।

13. `get` trap কখন চলে?

Proxy দিয়ে property access করলে।

14. `set` trap কখন চলে?

Proxy দিয়ে property assignment করলে।

15. `apply` trap কখন চলে?

Proxy target function call করলে।

16. Tagged template literal কী?

Template literal-এর আগে function/tag বসিয়ে template processing customize করার mechanism।

Example:

```
tag`Hello ${name}`;
```

95. Quick Revision

Symbol

```
const id = Symbol("id");
```

Unique:

```
Symbol("id") !== Symbol("id");
```

Global Symbol

```
const id = Symbol.for("id");
```

Retrieve:

```
Symbol.for("id") === id;
```

Registry key:

```
Symbol.keyFor(id);
```

Symbol Key

```
const key = Symbol("key");

const object = {
  [key]: 123
};
```

Get Symbol keys:

```
Object.getOwnPropertySymbols(object);
```

All own keys:

```
Reflect.ownKeys(object);
```

Symbol.toPrimitive

```
const object = {
  [Symbol.toPrimitive](hint) {
    return 100;
  }
};
```

Symbol.hasInstance

```
class User {
  static [Symbol.hasInstance](value) {
    return value?.role === "user";
  }
}
```

Symbol.species

```
class MyArray extends Array {  
  static get [Symbol.species]() {  
    return Array;  
  }  
}
```

Proxy

```
const proxy = new Proxy(target, {  
  get(target, property, receiver) {  
    return Reflect.get(  
      target,  
      property,  
      receiver  
    );  
  },  
  
  set(target, property, value, receiver) {  
    return Reflect.set(  
      target,  
      property,  
      value,  
      receiver  
    );  
  }  
});
```

Function Proxy

```
const proxy = new Proxy(fn, {  
  apply(target, thisArg, args) {  
    return Reflect.apply(  
      target,  
      thisArg,  
      args  
    );  
  }  
});
```

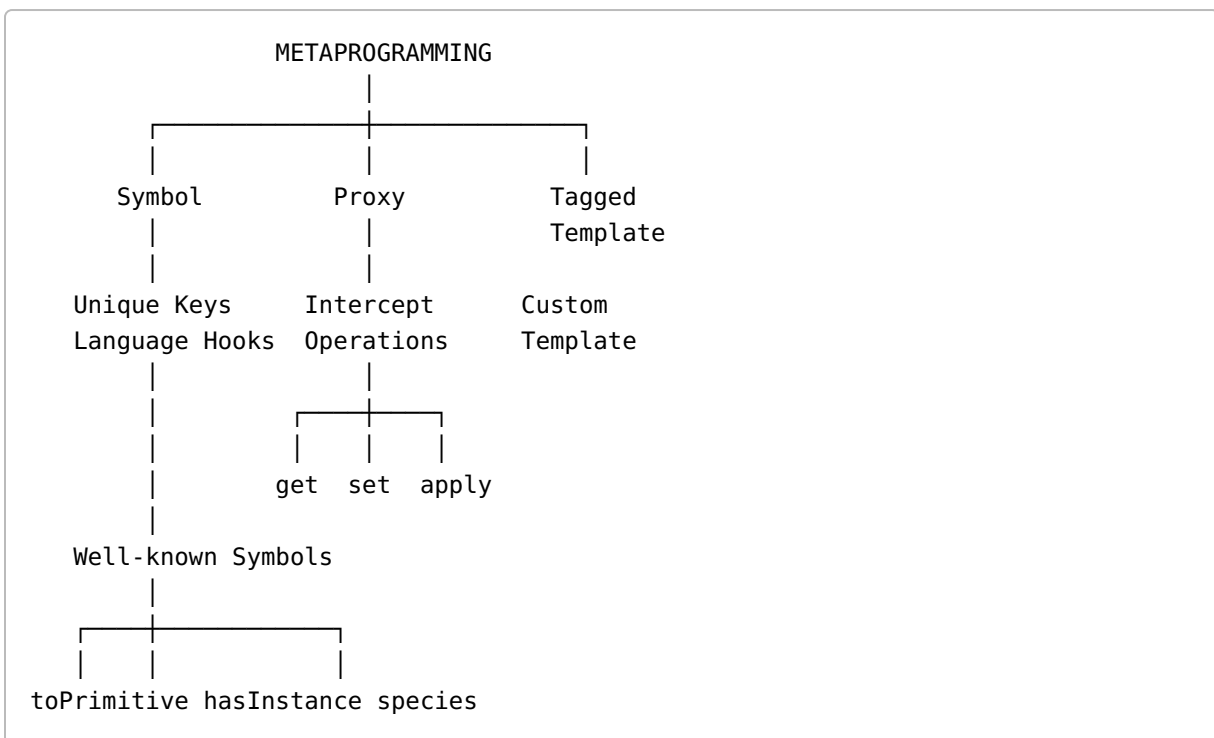
```
}  
});
```

Tagged Template

```
function tag(strings, ...values) {  
  // custom processing  
}  
  
tag`Hello ${name}`;
```

96. Final Mental Model

এই পুরো topic-টা এইভাবে মনে রাখো:



সবচেয়ে important relationships:

Symbol
↓

Unique identity + language hooks

Proxy

↓

Intercept behavior

Reflect

↓

Perform/delegate default behavior

Tagged Template

↓

Control template processing

Final Rules

`Symbol()` = **unique Symbol**

`Symbol.for()` = **shared registry Symbol**

`Symbol.keyFor()` = **registry key** বের করা

`Symbol.toPrimitive` = **object → primitive conversion control**

`Symbol.hasInstance` = `instanceof` **behavior control**

`Symbol.species` = **derived object creation constructor influence**

Proxy = operation intercept

Trap = interception hook

Reflect = low-level operation API / default behavior delegation

Tagged template = template literal processing customize করা

আর সবচেয়ে বড় mental model:

Normal JavaScript

↓

Language behavior happens automatically

Metaprogramming

↓

You get hooks to observe/customize
that behavior

এটাই JavaScript-এর metaprogramming-এর foundation ।